

start with random noise (cover entire sequence), iteratively
refine image.
↓
residuals from inception lab

Significant gains in inference to be made with more
radical architecture changes!

def quantization(): reduce precision of numbers
(less memory means higher latency/throughput, since inference
is memory-limited)

⇒ fp32 (4 bytes): needed for parameters and optimizer states during
training

bf16 (2 bytes): default for inference

fp32 (1 byte): can train it for dense.

int8 (1 byte): less accurate but cheaper than FP32, but inference
only

int4 (0.5 byte): cheaper, even less accurate.

Quantization-aware training: train with quantization, but doesn't

↓
post-training quantization (PTQ): run on sample data (scale up
to)

Activation-aware quantization: determine scale and zero point
for each layer or tensor
↓
select which weights (at-1%) to keep
in high precision based on activations

e.g. fp16 → int8 produces 4x lower memory

3.2x speedup

def model-pruning(): just rip out parts of an expensive model to
make it cheaper

Nvidia: Model pruning and distillation:

①: identify important (layer, head, hidden dimensions) on a small
calibration dataset (1024 samples)

②: Remove unimportant layers to get a smaller model

③: Distill the original model into pruned model.

Summary: reduce inference complexity without ~~losing~~ ^{accuracy}

From scratch recipe: 1: define faster model Architecture
2: train faster model

Distillation recipe: 1: define faster model Architecture
2: Initialize model weights using original model (which has a different architecture)
3: Repair faster model (distillation)

def speculative_sampling(): fast for inference

①: use a cheaper, draft model p to guess a few tokens

②: Evaluate with target model q (process tokens in-parallel), and accept if it looks good

in practice: Target model: 70B, draft model 8B

Target model: 8B, draft model 1B

Try to make draft model as close to target model (Distillation)

=> Extension to Draft model:

Medusa: draft model generates multiple tokens in

parallel: draft model takes high-level ^{parallel} features from target model

def continuous_batching():

Orca: A Distributed serving system for Transformer

Based Generative Models